

Assignment 1

Coding Environment Setup and Simple API Implementation

Applied Generative AI | Fall 2026

Part 1: FastAPI implementation

GitHub repository:

<https://github.com/xiangbuchu77/sps-genai-assignment1>

The project extends the Module 3 API with the spaCy word-embedding functionality from Module 2. It retains GET / and POST /generate and adds POST /embedding.

Embedding endpoint

```
POST /embedding Request: {"word": "apple"}
```

The response contains word, model, dimensions, and the full embedding. The implementation uses en_core_web_lg 3.8.0 and the classroom operation `nlp(input_word).vector`. The model is loaded once at startup, and the NumPy vector is converted to a JSON list. Tagging and parsing components are excluded because this operation only needs tokenization and static vectors.

Observed response: word = apple; model = en_core_web_lg; dimensions = 300.

First eight vector values (preview only): -0.36391, 0.43771, -0.20447, -0.22889, -0.14227, 0.27396, -0.01144, -0.18578

The complete 300-value response is saved in evidence/embedding_apple.json. Blank input, phrases, numbers, and punctuation return HTTP 422. An alphabetic token without a stored vector returns HTTP 404. Surrounding whitespace is removed; case is preserved.

Run and verify

```
uv sync --frozen

uv run fastapi dev app/main.py

uv run pytest -q

uv run python probability_solutions.py
```

Interactive documentation: <http://127.0.0.1:8000/docs>. The first installation downloads the model; requests then run locally. A Docker deployment is included and described on the next page.

Verification: 20 integration tests passed. Eight live HTTP checks confirmed the root route, documentation, OpenAPI schema, embedding output, error responses, and text generation. Vector outputs matched the actual model exactly. Evidence and dependency versions are included in the repository.

Part 1: Docker deployment

The repository includes Dockerfile, .dockerignore, pyproject.toml, and uv.lock. The image packages Python 3.12, the locked runtime dependencies, the application, and the en_core_web_lg model. The host only needs a running Docker installation; host Python and model files are not required.

1. Download the code and build

```
git clone https://github.com/xiangbuchu77/sps-genai-assignment1.git

cd sps-genai-assignment1

docker build -t sps-genai-assignment1 .
```

The first build needs internet access for the base image, Python dependencies, and approximately 382 MiB model download. Allow several minutes and adequate disk space.

2. Start the container

```
docker run --rm -d --name sps-genai-assignment1 \

-p 127.0.0.1:8000:80 sps-genai-assignment1

docker ps --filter name=sps-genai-assignment1
```

Wait for the health status to become healthy. Open <http://127.0.0.1:8000/docs> to query the API. The server listens on 0.0.0.0:80 inside the container; Docker maps host port 8000 to container port 80. If port 8000 is occupied, use 127.0.0.1:8001:80 and open port 8001 instead.

3. Query the embedding endpoint

```
curl -X POST http://127.0.0.1:8000/embedding \

-H 'Content-Type: application/json' \

-d '{"word": "apple"}'
```

Expected response: word = apple; model = en_core_web_lg; dimensions = 300; embedding = the full vector. GET / and POST /generate are also available in the container.

4. Logs and shutdown

```
docker logs sps-genai-assignment1

docker stop sps-genai-assignment1
```

Container verification

A Docker image was built successfully and tested on linux/arm64. The running container reported healthy. All 8 live HTTP checks passed, including the full 300-dimensional embedding, validation errors, and text generation. The embedding matched the locally verified vector. The test used no host directory mounts. Reproduction details are recorded in evidence/docker_verification.json.

Part 2: Rules of Probability

Question 1

Given $P(A) = 0.4$ and $P(B) = 0.3$, with A and B independent:

(a) Independence gives $P(A \cap B) = P(A)P(B) = 0.4 \times 0.3 = \mathbf{0.12}$.

(b) The addition rule gives $P(A \cup B) = P(A) + P(B) - P(A \cap B) = 0.4 + 0.3 - 0.12 = \mathbf{0.58}$.

Question 2

Given $P(A) = 0.5$, $P(B) = 0.4$, and $P(A | B) = 0.7$:

Independence would require $P(A | B) = P(A)$, since $P(B) > 0$. However, $0.7 \neq 0.5$. Therefore, **A and B are not independent**.

Equivalently, $P(A \cap B) = P(A | B)P(B) = 0.7 \times 0.4 = 0.28$, while $P(A)P(B) = 0.5 \times 0.4 = 0.20$. These are unequal.

Question 3

Given $P(A) = 0.6$, $P(B | A) = 0.5$, and $P(B | A^c) = 0.2$:

$P(A^c) = 1 - 0.6 = 0.4$.

By the law of total probability:

$P(B) = P(B | A)P(A) + P(B | A^c)P(A^c) = 0.5 \times 0.6 + 0.2 \times 0.4 = 0.30 + 0.08 = 0.38$.

By Bayes' rule:

$P(A | B) = P(B | A)P(A) / P(B) = 0.30 / 0.38 = 15/19 \approx \mathbf{0.789474}$, or **78.95%**.

Question 4

Let D denote having the disease and + denote a positive test result. The given rates are $P(D) = 0.02$, $P(+ | D) = 0.95$, and $P(- | D^c) = 0.90$.

Thus, $P(D^c) = 0.98$ and the false-positive rate is $P(+ | D^c) = 1 - 0.90 = 0.10$.

$P(+) = P(+ | D)P(D) + P(+ | D^c)P(D^c) = 0.95 \times 0.02 + 0.10 \times 0.98 = 0.019 + 0.098 = 0.117$.

$P(D | +) = P(+ | D)P(D) / P(+) = 0.019 / 0.117 = 19/117 \approx \mathbf{0.162393}$, or **16.24%**.

For example, among 10,000 people, the expected counts are 200 with the disease and 9,800 without it. There would be 190 true positives and 980 false positives. Consequently, $190 / (190 + 980) \approx 16.24\%$ of positive results would be true positives. The low disease prevalence explains why this is much lower than the 95% true-positive rate.

Question 5

x	P(X = x)	xP(X = x)	x ² P(X = x)
85	0.375	31.875	2709.375
90	0.375	33.750	3037.500
95	0.125	11.875	1128.125
100	0.125	12.500	1250.000
Total	1.000	90.000	8125.000

(a) $E[X] = \sum xP(X = x) = 31.875 + 33.750 + 11.875 + 12.500 = \mathbf{90}$.

(b) $E[X^2] = \sum x^2P(X = x) = 8125$.

$\text{Var}(X) = E[X^2] - (E[X])^2 = 8125 - 90^2 = 8125 - 8100 = \mathbf{25}$ (score points squared).

(c) For the sample {85, 90, 85, 95, 90, 85, 100, 90}:

Sample mean = $(85 + 90 + 85 + 95 + 90 + 85 + 100 + 90) / 8 = 720 / 8 = \mathbf{90}$.

The sample mean equals $E[X]$ for this particular sample. Its relative frequencies are 3/8, 3/8, 1/8, and 1/8, exactly matching the specified probabilities. Other random samples need not have exactly the same mean.

Question 6

(a) With probabilities (0.4, 0.3, 0.2, 0.1), entropy in bits is:

$$\begin{aligned} H(X) &= -\sum p(x) \log_2 p(x) \\ &= -[0.4 \log_2(0.4) + 0.3 \log_2(0.3) + 0.2 \log_2(0.2) + 0.1 \log_2(0.1)] \\ &= 0.528771 + 0.521090 + 0.464386 + 0.332193 \\ &\approx \mathbf{1.846439 \text{ bits}}. \end{aligned}$$

(b) If the four messages are equally likely, each has probability 1/4:

$$H(X) = -4 \times (1/4) \log_2(1/4) = -\log_2(1/4) = \mathbf{2 \text{ bits}}.$$

The uniform distribution has the maximum entropy for four possible messages. Making the messages equally likely increases the entropy by approximately $2 - 1.846439 = \mathbf{0.153561 \text{ bits}}$.

References

Assignment1-1.pdf; Module 2 Practice 2: Word Sampling; Module 2 Practice 3: Word Embeddings; Module 3 Activity: First Docker/FastAPI Project Setup and Simple Text Generator.

[spaCy vectors and similarity](#)

[FastAPI testing](#)

[uv Docker integration](#)